

CE 6320 Data Structures

Sumit Kumar - dal833771

Homework 1

1 Problem 1

Listing 1: Problem1.h

```
1
2 #include <iostream>
3 using namespace std;
4
5 struct Node {
6     int data;
7     Node* next;
8 };
9
10 class CircularLinkedList {
11     Node* front;
12     Node* back;
13     Node* current;
14
15 public:
16     CircularLinkedList() {
17         front = nullptr;
18         back = nullptr;
19         current = nullptr;
20         back->next = front;
21     }
22
23     ~CircularLinkedList() {}
24
25     void NEXT(int k);
26     void DEL();
27     /*!
28         Builds a circular list of 1..n nodes, returns pointer
29         to head,
30         initializes cur appropriately, and sets size = n.
31     */
32     Node* buildCircle(int n, Node*& cur, int& size);
33 }
```

```

33     // Advances cur by k % size steps.
34     void nextK(Node*& cur, int& size, int k);
35
36     // Deletes the node at cur, updates links, and sets cur
37     // = next node.
38     void delCurrent(Node*& cur, int& size);
39
40     // Prints the list once around starting from cur (or
41     // EMPTY if size = 0).
42     void printOnce(Node* cur);
43 };

```

Listing 2: Problem1.cpp

```

1  #include "Problem1.h"
2
3  CircularLinkedList::CircularLinkedList() {}
4
5  void CircularLinkedList::NEXT(int k) {
6      for (int i = 0; i < k; i++) {
7          current = current->next;
8      }
9  }
10
11 void CircularLinkedList::DEL() {
12     if (current == nullptr)
13         return;
14
15     // If the list only has 1 node
16     if (current == front && current == back) {
17         delete current;
18         front = nullptr;
19         back = nullptr;
20         current = nullptr;
21         return;
22     }
23
24     // Save the node to be deleted, and next node to move
25     // current to
26     Node* victim = current;
27     Node* nextNode = victim->next;
28
29     // Find the node before current
30     Node* prevNode = front;
31     if (victim != front) {
32         while (prevNode->next != victim)
33             prevNode = prevNode->next;
34     }
35
36     // If the list has multiple nodes

```

```

36     if (victim == front) {
37         front = nextNode;
38         back->next = front;
39     }
40     else {
41         prevNode->next = nextNode;
42     }
43
44     if (victim==back) {
45         back = prevNode;
46     }
47
48     current = nextNode;
49     delete victim;
50 }
51
52 // Builds a circular list of 1..n nodes, returns pointer to
53 // head,
54 // initializes cur appropriately, and sets size = n.
55 Node* CircularLinkedList::buildCircle(int n, Node*& cur, int
56 & size) {
57     front = new Node;
58     front->data = 1;
59     front->next = front; // Point to itself to become
60                          // circular
61
62     back = front;
63
64     for (int i = 2; i <= n; i++) {
65         Node* newNode = new Node;
66         newNode->data = i;
67         newNode->next = front; // Point to front for
68                                // circular
69         back->next = newNode; // The last node should point
70                                // to new one
71
72         back = newNode;
73     }
74     cur = front; // Initialize current to front
75     size = n;
76     return front;
77 }
78
79 // Advances cur by k % size steps.
80 void CircularLinkedList::nextK(Node*& cur, int& size, int k)
81 {
82     int steps = k % size;
83     for (int i = 0; i < steps; i++) {
84         cur = cur->next;
85     }

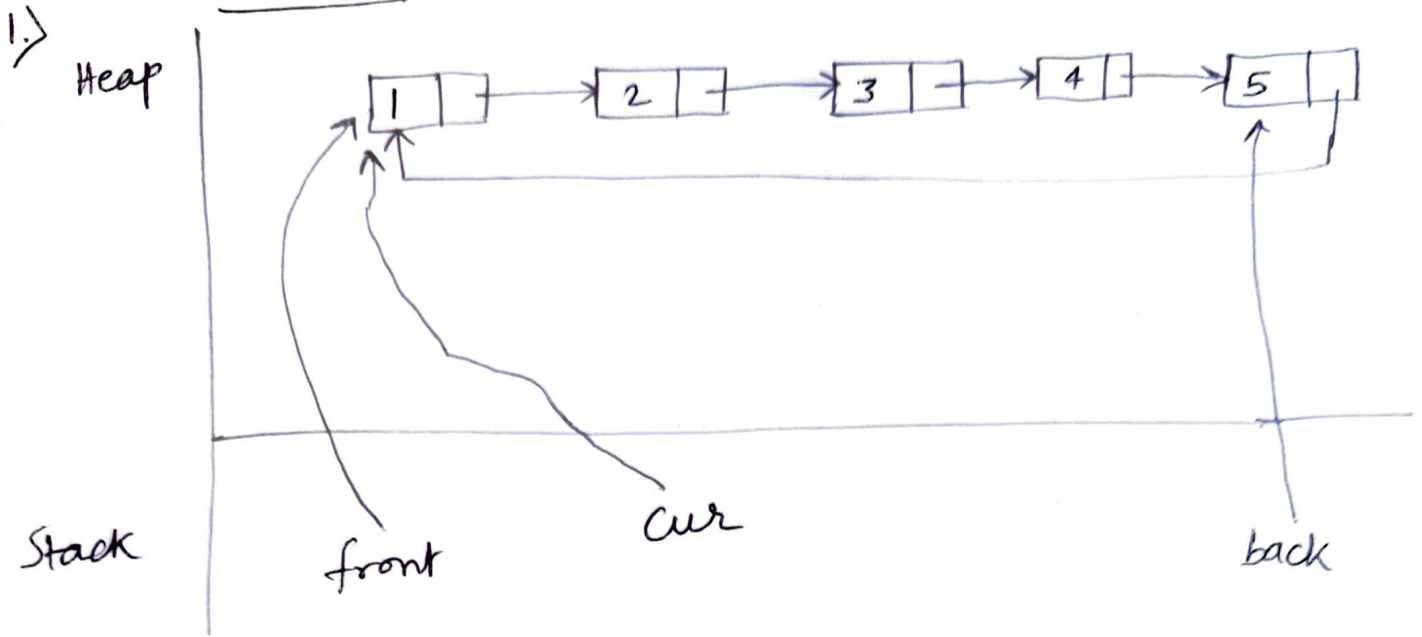
```

```

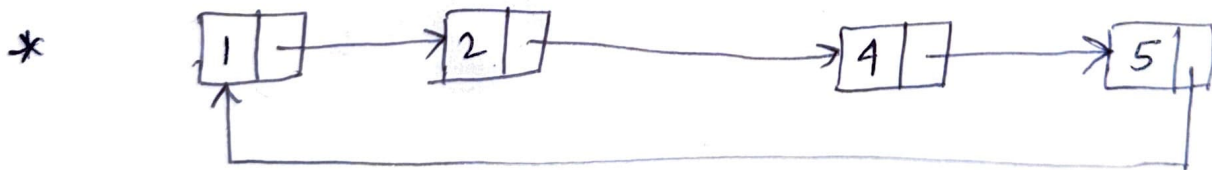
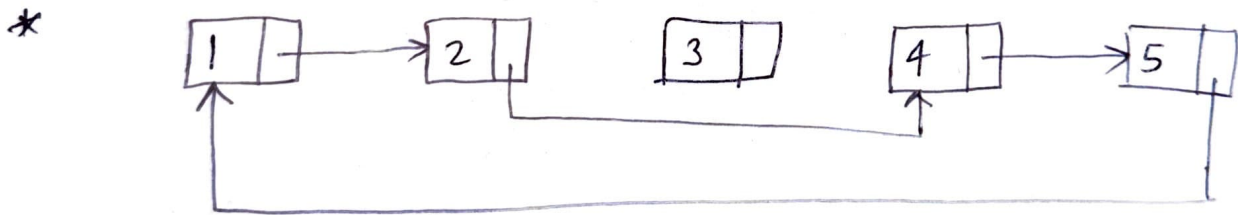
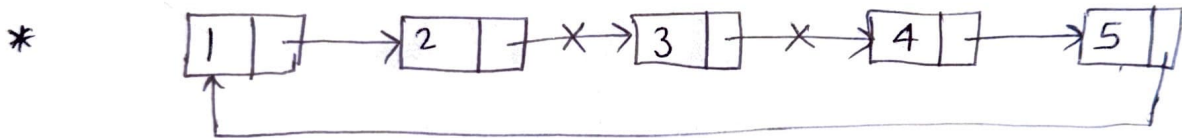
80 }
81
82
83 // Deletes the node at cur, updates links, and sets cur =
   next node.
84 void CircularLinkedList::delCurrent(Node*& cur, int& size) {
85     if (cur == nullptr || size == 0)
86         return;
87     if (size == 1) {
88         delete cur;
89         cur = nullptr;
90         size--;
91         return;
92     }
93     else if (size > 1) {
94         Node* nextNode = cur->next;
95         Node* prevNode = cur;
96
97         // Find previous node
98         Node* temp = nextNode;
99         while (temp->next != cur)
100             temp = temp->next;
101
102         prevNode = temp;
103         delete cur;
104         cur = nextNode;
105         size--;
106     }
107 }
108
109 // Prints the list once around starting from cur (or EMPTY
   if size == 0).
110 void CircularLinkedList::printOnce(Node* cur) {
111     Node* temp = cur;
112     if (cur == nullptr) {
113         cout << "EMPTY" << endl;
114         return;
115     }
116     while (cur != nullptr) {
117         cout << cur->data << "□";
118         cur = cur->next;
119         if (cur == temp)
120             break;
121     }
122 }

```

Problem 1 :

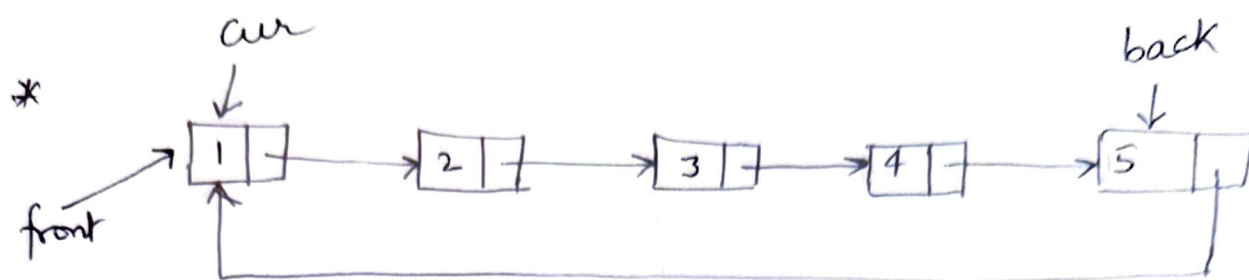


When node 3 is deleted :

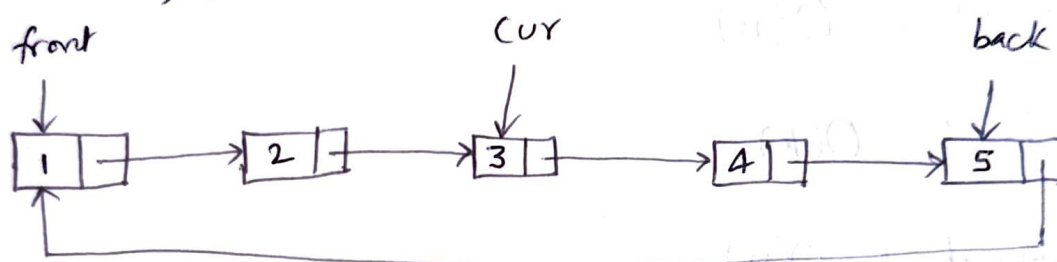


Problem 1:

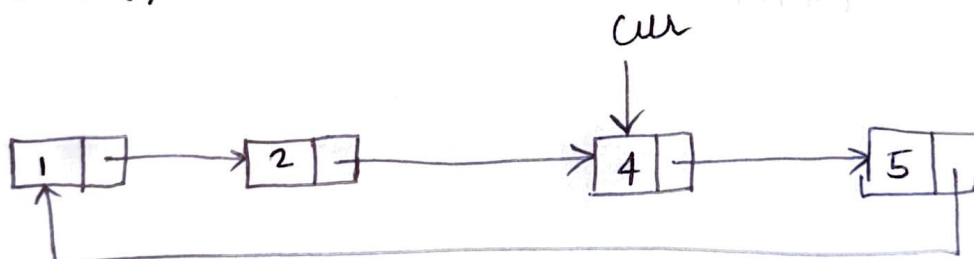
3)



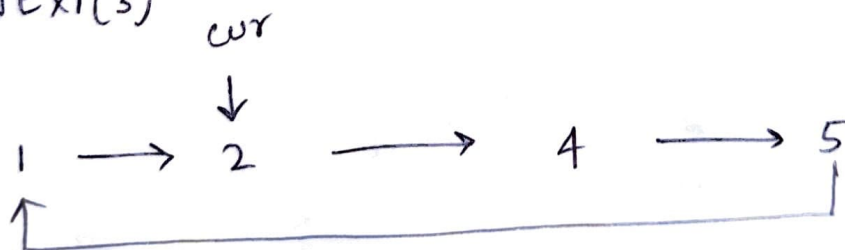
* NEXT(2):



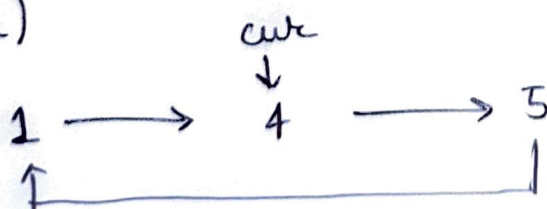
* DEL()



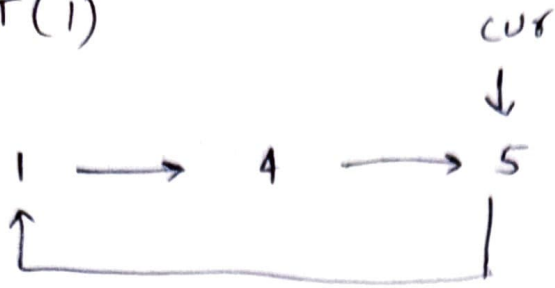
* NEXT(3)



* DEL()



* NEXT(1)



④ Time Complexity :

* Build Circle : $O(n)$

* nextK : $O(k)$

* delCurrent : $O(n)$

* printOnce : $O(n)$

2 Problem 2

Listing 3: Problem2.h

```
1  #include <iostream>
2  using namespace std;
3
4  /*!
5   * Problem 2: Unrolled Linked List [20 points]
6   */
7
8
9  const int B = 4;
10
11 struct Block {
12     int a[B]; // items packed in a[0..cnt-1]
13     int cnt; // number of valid items in this block
14     Block* next; // pointer to next block
15 };
16
17 // Unrolled linked list
18 struct UList {
19     Block* head; // head block (nullptr if empty)
20     int n; // total number of items
21     UList(): head(nullptr), n(0) {}
22 };
23
24 // Build list from arr[0..n-1], filling blocks left-to-right
25
26
27 void buildFromArray(const int* arr, int n, UList& L);
28
29 // Return the value at global index idx.
30 int getAt(const UList& L, int idx);
31
32 // Insert x at global index idx, shifting items as needed.
33 // If a block is full, split it.
34 void insertAt(UList& L, int idx, int x);
35
36 // Erase the element at global index idx, shifting items as
37 // needed.
38 // If a block becomes under half full, borrow or merge with
39 // neighbor.
40 void eraseAt(UList& L, int idx);
41
42 // Helper: find block containing index idx, return pointer
43 // and local offset.
44 Block* findBlock(UList& L, int idx, int& off);
```

Listing 4: Problem2.cpp

```
1  #include "Problem2.h"
```

```

2
3 // Build list from arr[0..n-1], filling blocks left-to-right
.
4 void buildFromArray(const int* arr, int n, UList& L) {
5     Block* current = new Block;
6     current->cnt = 0;
7     current->next = nullptr;
8
9     L.head = current;
10    L.n = n;
11
12    for (int i = 0; i < n; i++) {
13        // When the current block is full
14        if (current->cnt == B) {
15            Block* newBlock = new Block;
16            newBlock->cnt = 0;
17            newBlock->next = nullptr;
18
19            current->next = newBlock;
20            current = newBlock;
21        }
22        current->a[current->cnt] = arr[i];
23        current->cnt++;
24    }
25 }
26
27 // Return the value at global index idx.
28 int getAt(const UList& L, int idx) {
29     int offset = 0;
30     Block* cur = L.head;
31
32     while(cur != nullptr) {
33         if (idx < offset + cur->cnt) {
34             int local_idx = idx - offset;
35             return cur->a[local_idx];
36         }
37         offset += cur->cnt;
38         cur = cur->next;
39     }
40
41     return -1; // invalid index
42 }
43
44 // Insert x at global index idx, shifting items as needed.
45 // If a block is full, split it.
46 void insertAt(UList& L, int idx, int x) {
47     int offset = 0;
48     Block* cur = L.head;
49
50     while (cur!= nullptr && (offset + cur->cnt < idx)) {

```

```

51         offset += cur->next;
52         cur = cur->next;
53     }
54
55     // If idx is at end, create new block at tail
56     if (cur == nullptr) {
57         Block* tail = L.head;
58         while (tail->next != nullptr) {
59             tail = tail->next;
60         }
61
62         cur = new Block;
63         cur->cnt = 0;
64         cur->next = nullptr;
65         tail->next = cur;
66         offset = L.n;
67     }
68
69     // If the block is full, split it
70     if (cur->cnt == B) {
71         int mid = B/2;
72         Block* right = new Block;
73         right->cnt = 0;
74         right->next = cur->next;
75
76         //move upper half to new block
77         for (int i = 0; i < (B-mid); i++) {
78             right->a[i] = cur->a[mid+i];
79         }
80         right->cnt = B - mid;
81         cur->cnt = mid;
82
83         // Decide which side the insertion belongs to
84         if (idx >= offset + mid) {
85             offset += mid;
86             cur = right;
87         }
88     }
89
90     // local index inside the chosen block
91     int local = idx - offset;
92
93     // shift elements right to make room
94     for (int i = cur->cnt; i > local; i--) {
95         cur->a[i] = cur->a[i-1];
96     }
97
98     cur->a[local] = x;
99     cur->cnt++;
100    L.n++;

```

```

101 }
102
103 // Erase the element at global index idx, shifting items as
    needed.
104 // If a block becomes under half full, borrow or merge with
    neighbor.
105 void eraseAt(UList& L, int idx) {
106     if (L.head == nullptr || idx < 0 || idx >= L.n) {
107         return;
108     }
109
110     int offset = 0;
111     Block* cur = L.head;
112
113     while (cur != nullptr) {
114         if (idx < offset + cur->cnt) {
115             int local_idx = idx - offset;
116
117             // shift left to remove element
118             for (int i = local_idx; i < cur->cnt - 1; i++) {
119                 cur->a[i] = cur->a[i+1];
120             }
121
122             cur->cnt--;
123             L.n--;
124
125             // Rebalance if block is too empty
126             if (cur->cnt < B/2) {
127                 Block* next = cur->next;
128
129                 if (next != nullptr && next->cnt > B/2) {
130                     // Borrow one item from next block
131                     cur->a[cur->cnt] = next->a[0];
132
133                     for (int i = 0; i < next->cnt - 1; i++) {
134                         {
135                             next->a[i] = next->a[i+1];
136                         }
137
138                         next->cnt--;
139                         cur->cnt++;
140                     }
141                 }
142                 else if (next != nullptr) {
143                     // merge with next block
144                     for (int i = 0; i < next->cnt; i++) {
145                         cur->a[cur->cnt++] = next->a[i];
146                     }
147
148                     Block* temp = next;
149                     cur->next = next->next;

```

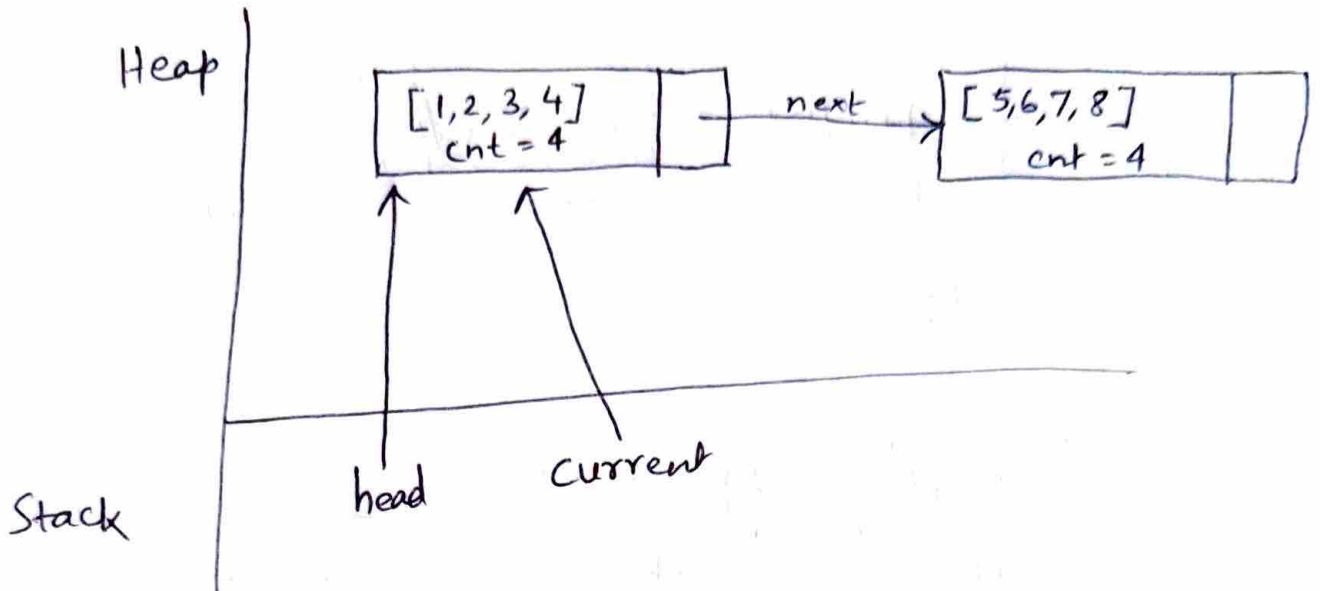
```

148             delete temp;
149         }
150     }
151     return;
152 }
153
154     offset += cur->cnt;
155     cur = cur->next;
156 }
157 }
158
159 // Helper: find block containing index idx, return pointer
160 // and local offset.
161 Block* findBlock(UList& L, int idx, int& off) {
162     int offset = 0;
163     Block* cur = L.head;
164
165     while (cur != nullptr) {
166         if (idx < offset + cur->cnt) {
167             off = idx - offset;
168             return cur;
169         }
170         offset += cur->cnt;
171         cur = cur->next;
172     }
173
174     off = -1;
175     return nullptr;
176 }

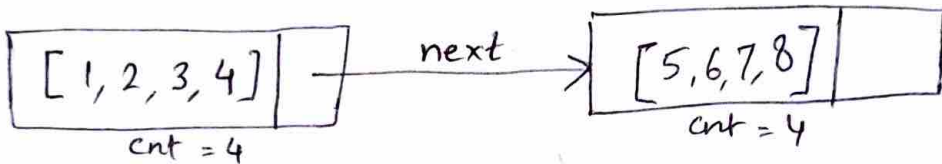
```

Problem 2:

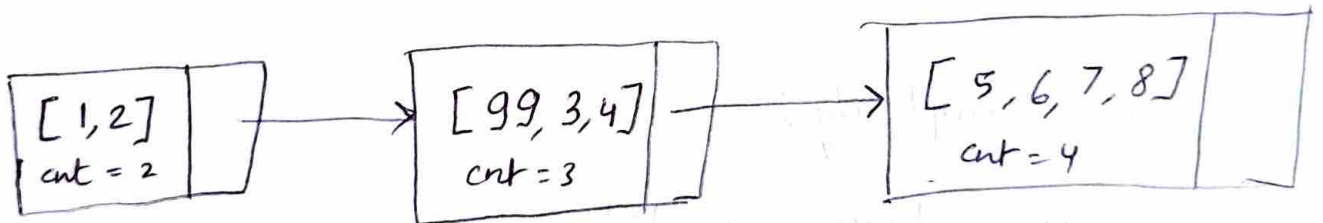
1.)



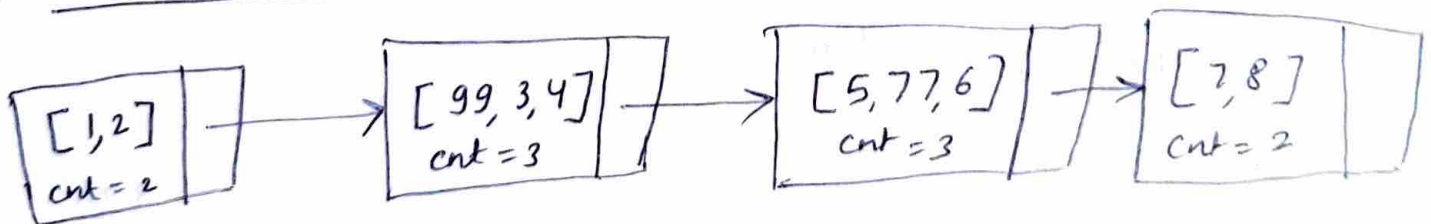
3.)



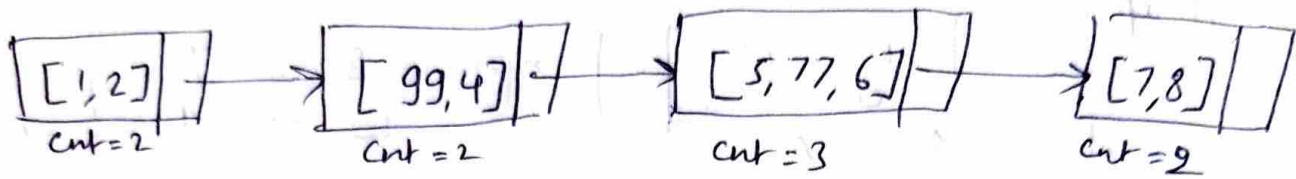
* insertAt(2, 99) :



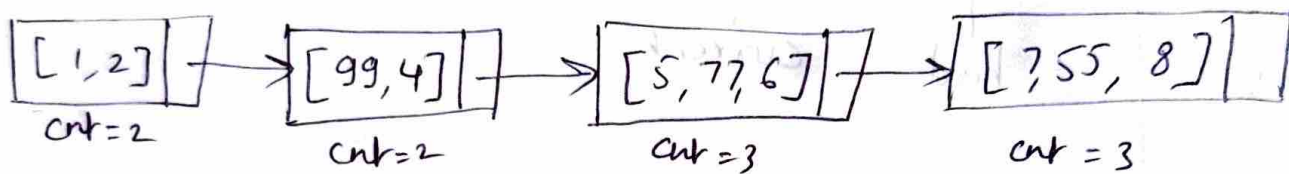
* insertAt(6, 77) :



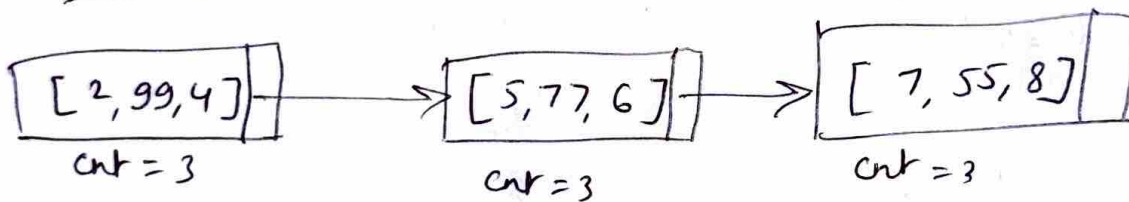
* eraseAt(3):



* insertAt(8, 55):



* eraseAt(0):



4) Time Complexity:

buildFromArray : $O(n)$
getAt : $O(n)$
insertAt : $O(n)$
eraseAt : $O(n)$

3 Problem 3

Listing 5: Problem3.h

```
1  /*!
2      Problem 3: LRU Cache [20 points]
3  */
4
5  #include <iostream>
6  using namespace std;
7
8  struct Node {
9      int key, val;
10     Node *prev, *next;
11     Node(int k, int v): key(k), val(v), prev(nullptr), next(
12         nullptr) {}
13 };
14
15 struct LRU {
16     int cap; // capacity C
17     int size; // current number of keys in cache
18     int K; // key range upper bound (keys in [0, K))
19     Node **dir; // array directory: dir[key] = Node* or
20                 nullptr
21     Node *head; // most recently used (MRU)
22     Node *tail; // least recently used (LRU)
23     LRU(): cap(0), size(0), K(0), dir(nullptr), head(nullptr),
24         tail(nullptr) {}
25 };
26
27 // Initialize the cache with capacity Ccap and key range K.
28 // Allocate dir on the heap and set all entries to nullptr.
29 void initLRU(LRU &C, int Ccap, int K);
30
31 // Return true and set outVal if present (and move node to
32 // MRU); else return false.
33 bool get(LRU &C, int key, int &outVal);
34
35 // Insert or update (key, val). If inserting and size==cap,
36 // evict LRU.
37 // Move the (possibly new) key to MRU position
38 void put(LRU &C, int key, int val);
39
40 // (Helpers students should implement/and may call from get/
41 // put)
42 // Remove node u from list in O(1).
43 void detach(LRU &C, Node *u);
44
45 // Insert node u at head (MRU) in O(1).
46 void pushFront(LRU &C, Node *u);
```

```

41
42 // Evict the LRU (tail) node in O(1); update dir and size;
    delete the node.
43 void evictLRU(LRU &C);

```

Listing 6: Problem3.cpp

```

1  #include "Problem3.h"
2
3  // Initialize the cache with capacity Ccap and key range K.
4  // Allocate dir on the heap and set all entries to nullptr.
5  void initLRU(LRU &C, int Ccap, int K) {
6      C.cap = Ccap;
7      C.K = K;
8      C.size = 0;
9      C.dir = new Node*[K];
10     for (int i = 0; i < K; i++) {
11         C.dir[i] = nullptr;
12     }
13 }
14
15 // Return true and set outVal if present (and move node to
    MRU); else return false.
16 bool get(LRU &C, int key, int &outVal) {
17     if (key < 0 || key >= C.K || C.dir[key] == nullptr)
18         return false;
19     outVal = C.dir[key]->val;
20
21     // Move the accessed node to front (MRU)
22     detach(C, C.dir[key]);
23     pushFront(C, C.dir[key]);
24     return true;
25 }
26
27 // Insert or update (key, val). If inserting and size==cap,
    evict LRU.
28 // Move the (possibly new) key to MRU position
29 void put(LRU &C, int key, int val) {
30     if (key < 0 || key > C.K)
31         return;
32     if (C.dir[key] != nullptr) {
33         // key exists, update value and move to front
34         C.dir[key]->val = val;
35         detach(C, C.dir[key]);
36         pushFront(C, C.dir[key]);
37     } else {
38         // key does not exist, insert new node
39         if (C.size == C.cap) {
40             evictLRU(C);
41         }

```

```

42         Node* newNode = new Node(key, val);
43         pushFront(C, newNode);
44         C.dir[key] = newNode;
45         C.size++;
46     }
47 }
48
49 // (Helpers students should implement/and may call from get/
    put)
50 // Remove node u from list in O(1).
51 void detach(LRU &C, Node *u) {
52     if (u == nullptr) return;
53     if (u->prev != nullptr) // u is not head node
54         u->prev->next = u->next;
55     else // u is head node
56         C.head = u->next;
57     if (u->next != nullptr) // u is not a tail node
58         u->next->prev = u->prev;
59     else // u is a tail node
60         C.tail = u->prev;
61     u->prev = nullptr;
62     u->next = nullptr;
63 }
64
65 // Insert node u at head (MRU) in O(1).
66 void pushFront(LRU &C, Node *u) {
67     if (u == nullptr) return;
68     u->next = C.head;
69     u->prev = nullptr;
70
71     if (C.head != nullptr) // non-empty list
72         C.head->prev = u;
73     else // empty list
74         C.tail = u;
75
76     C.head = u;
77 }
78 }
79
80 // Evict the LRU (tail) node in O(1); update dir and size;
    delete the node.
81 void evictLRU(LRU &C) {
82     if (C.tail == nullptr) return;
83
84     Node* oldTail = C.tail;
85
86     //remove from directory
87     C.dir[oldTail->key] = nullptr;
88
89     // remove from DLL

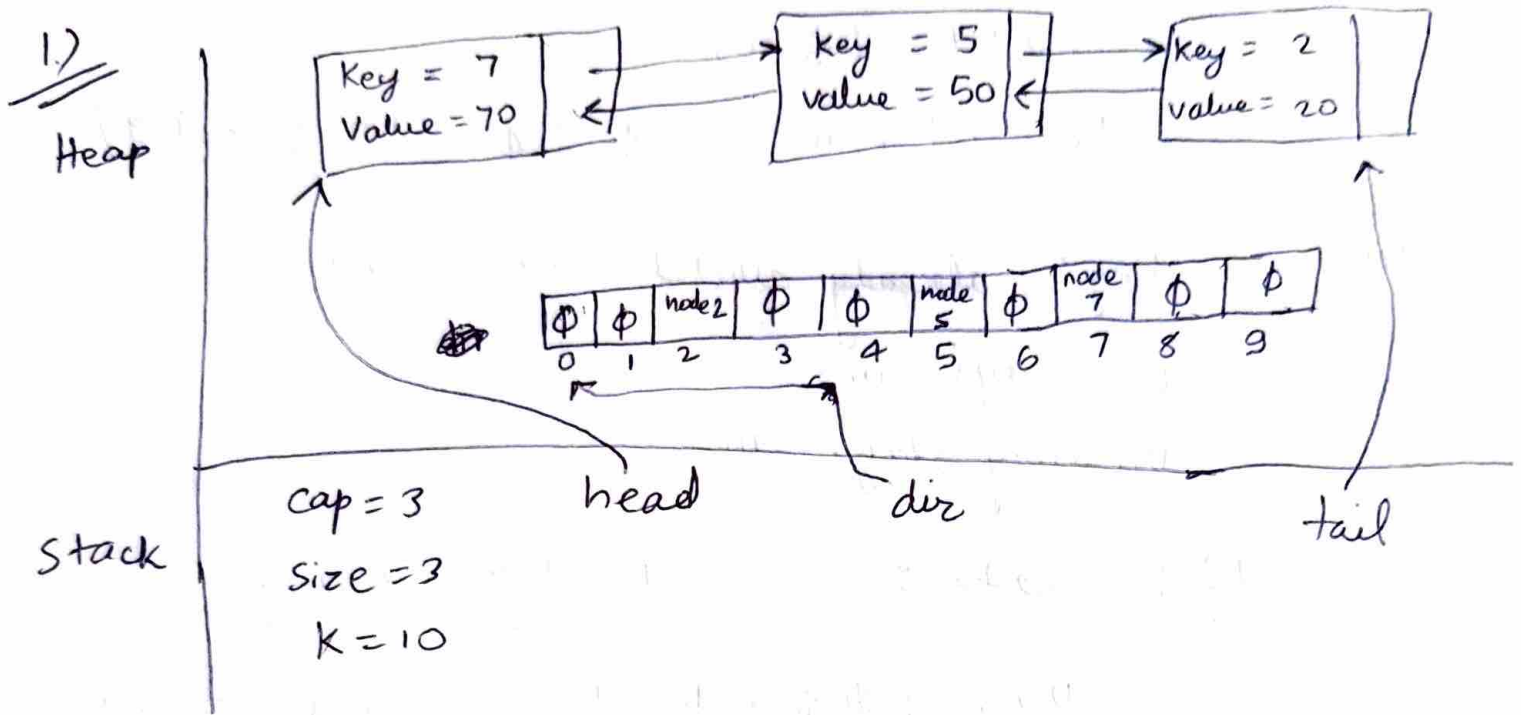
```

```

90     if (oldTail->prev != nullptr) {          // More tahn 1 node
91         present
92         oldTail->prev->next = nullptr;
93         C.tail = oldTail->prev;
94     } else {                                  // Only 1 node presetn
95         C.head = nullptr;
96         C.tail = nullptr;
97     }
98     //update size
99     C.size--;
100
101     delete oldTail;
102
103 }

```

Problem 3 :



3) put(2, 20) : LRU Cache : [2:20]
Dir : [$\phi, \phi, \text{node}_2, \phi, \dots, \phi$]

put(5, 50) : Cache : [5:50, 2:20]
Dir : [$\phi, \phi, \text{node}_2, \phi, \phi, \text{node}_5, \dots, \phi$]

put(7, 70) : Cache : [7:70, 5:50, 2:20]
Dir : [$\phi, \phi, \text{node}_2, \phi, \phi, \text{node}_5, \phi, \text{node}_7, \dots, \phi$]

get(2) : Cache : [2:20, 7:70, 5:50]

Nothing evicted
Directory unchanged.

put(9, 90) : Cache : [9:90, 2:20, 7:70]

Evicted : (5:50)

Dir : [ϕ , ϕ , node 2, ϕ , ϕ , ϕ , ϕ , node 7, ϕ , node 9]

get(5) : Key 5 already evicted, not found

List stays same

Directory stays same

put(2, 21) : Cache : [2:21, 9:90, 7:70]

Dir : [ϕ , ϕ , node 2, ϕ , ϕ , ϕ , ϕ , node 7, ϕ , node 9]

4) Time Complexity :

InitLRU : It iterates K times -

$O(K)$

get() : $O(1)$

As directory lookup is direct
Removing & reinserting node is also $O(1)$

put() : $O(1)$

If key exists, update & move to front in $O(1)$
If key is new, evict & insert in $O(1)$

4 Problem 4

Listing 7: Problem4.h

```
1  /*
2      Problem 4: Polynomial Representation & Recursive
          Evaluation [20 points]
3  */
4
5  #include <iostream>
6  using namespace std;
7
8  struct Term {
9      double coef;
10     int exp;
11     Term* next;
12     Term(double c, int e): coef(c), exp(e), next(nullptr) {}
13 };
14
15 // Build a polynomial from arrays coef[i], exp[i], i=0..m-1.
16 // Insert terms one-by-one so that the final list is
17 // strictly descending in exp
18 // and contains no duplicate exponents (combine coeffs; drop
19 // zero terms).
20 Term* buildFromArrays(const double* coef, const int* exp,
21     int m);
22 // Insert (c,e) into the sorted list (descending exponents).
23 // Combine coefficients if exp already exists; remove node
24 // if coef becomes 0.
25 void insertTerm(Term*& head, double c, int e);
26 // Remove the term with exponent e, if present.
27 void eraseExp(Term*& head, int e);
28 // Recursively compute x^k for k>=0 using fast
29 // exponentiation.
30 // Must run in O(log k).
31 double powInt(double x, int k);
32
33 // Recursive evaluation of the polynomial at x.
34 // Assume list is strictly descending in exp with no
35 // duplicates.
36 // Hint: if current node is (c,e) and next is (c2,e2),
37 // multiply the running
38 // value by x^(e - e2) before adding c2, and recurse.
39 double evalRecursive(const Term* head, double x);
```

Listing 8: Problem4.cpp

```
1  #include "Problem4.h"
2
3  // Build a polynomial from arrays coef[i], exp[i], i=0..m-1.
```

```

4 // Insert terms one-by-one so that the final list is
  // strictly descending in exp
5 // and contains no duplicate exponents (combine coeffs; drop
  // zero terms).
6 Term* buildFromArrays(const double* coef, const int* exp,
  int m) {
7     Term* head = nullptr;
8
9     for (int i = 0; i < m; i++) {
10         if (coef[i] == 0) continue; // skip zero coeff
11         insertTerm(head, coef[i], exp[i]);
12     }
13     return head;
14 }
15
16 // Insert (c,e) into the sorted list (descending exponents).
17 // Combine coefficients if exp already exists; remove node
  // if coef becomes 0.
18 void insertTerm(Term*& head, double c, int e) {
19     if (c == 0) return;
20
21     Term* prev = nullptr;
22     Term* cur = head;
23
24     // Find insertion point
25     while (cur != nullptr && cur->exp > e) {
26         prev = cur;
27         cur = cur->next;
28     }
29
30     // same exponent present
31     if (cur != nullptr && cur->exp == e) {
32         cur->coef += c;
33
34         // If coeff becomes 0
35         if (cur->coef == 0) {
36             if (prev == nullptr) {
37                 head = cur->next;
38             }
39             else {
40                 prev->next = cur->next;
41             }
42             delete cur;
43         }
44         return;
45     }
46
47     // Insert new term
48     Term* newTerm = new Term(c, e);
49     newTerm->next = cur;

```

```

50
51     if (prev == nullptr)
52         head = newTerm;
53     else
54         prev->next = newTerm;
55 }
56
57 // Remove the term with exponent e, if present.
58 void eraseExp(Term*& head, int e) {
59     Term* prev = nullptr;
60     Term* cur = head;
61
62     while (cur != nullptr && cur->exp > e) {
63         prev = cur;
64         cur = cur->next;
65     }
66
67     if (cur != nullptr && cur->exp == e) {
68         if (prev == nullptr) {
69             head = cur->next;
70         }
71         else {
72             prev->next = cur->next;
73         }
74         delete cur;
75     }
76 }
77
78 // Recursively compute x^k for k>=0 using fast
79 // exponentiation.
80 // Must run in O(log k).
81 double powInt(double x, int k) {
82     if (k == 0) return 1;
83
84     double half = powInt(x, k/2);
85
86     if (k%2 == 0)
87         return half * half;
88     else
89         return x * half * half;
90 }
91
92 // Recursive evaluation of the polynomial at x.
93 // Assume list is strictly descending in exp with no
94 // duplicates.
95 // Hint: if current node is (c,e) and next is (c2,e2),
96 // multiply the running
97 // value by x^(e - e2) before adding c2, and recurse.
98 double evalRecursive(const Term* head, double x) {
99     if (head == nullptr) return 0;

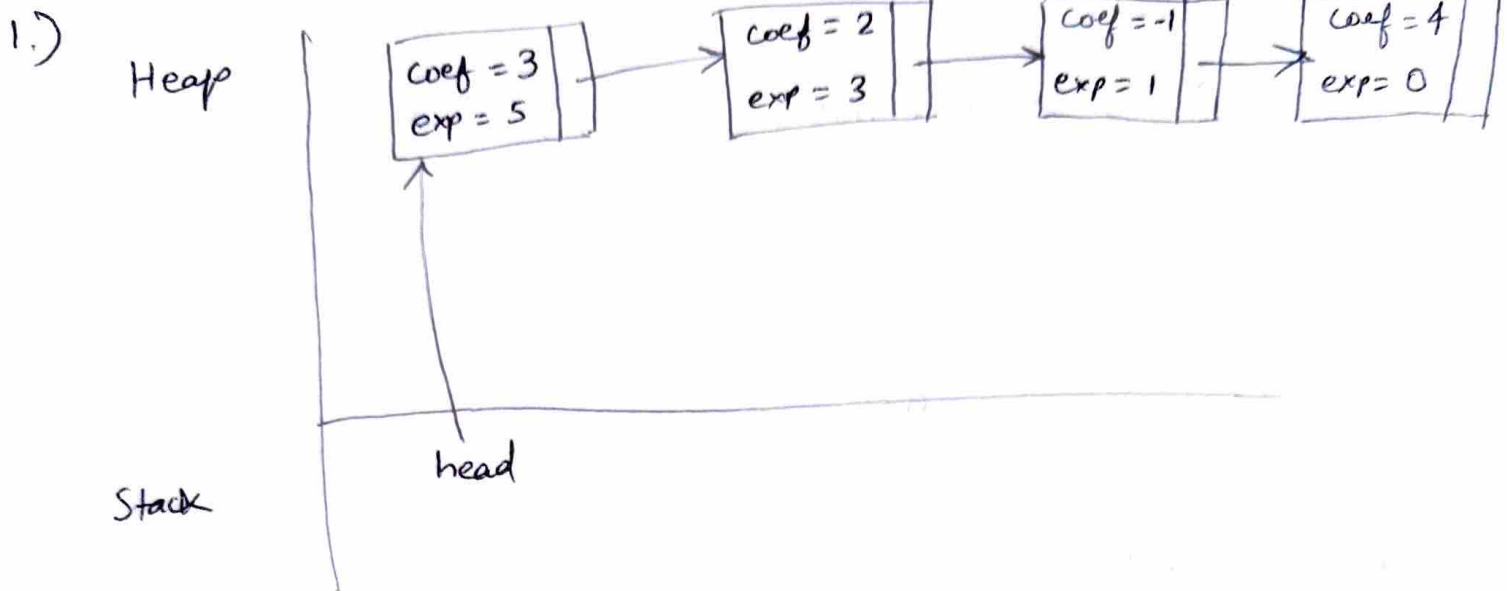
```

```

97
98     // Base case: single term
99     if (head->next == nullptr)
100         return head->coef * powInt(x, head->exp);
101
102     // Recursive case: current term + next terms
103     return head->coef * powInt(x, head->exp) + evalRecursive
104         (head->next, x);
105 }

```

Problem 4:



3.) $P(x) = 3x^5 + 2x^3 - x + 4$

* insertTerm (+5, 3):

$$(3, 5) \rightarrow (7, 3) \rightarrow (-1, 1) \rightarrow (4, 0)$$

↑
node combined

* eraseExp(1):

$$(3, 5) \rightarrow (7, 3) \rightarrow (4, 0)$$

node (-1, 1) removed

* insertTerm (-3, 5):

$$(7, 3) \rightarrow (4, 0)$$

node (3, 5) combined with (-3, 5) and removed

* ~~when x=0~~ Final Polynomial $P(x) = 7x^3 + 4$

* When $x=2$:

$(7, 3) \longrightarrow (4, 0)$

$$\begin{aligned} & 7 \times 2^3 + \text{evalRecursive}(4, 0, 2) \\ = & 7 \times 8 + 4 \times 2^0 \\ = & 56 + 4 \times 1 \\ = & 56 + 4 = 60 \end{aligned}$$

4.) Time Complexity :

* buildFromArrays : Each insertion can take $O(n)$
And there are n nodes.

So, Time complexity is $O(n^2)$

* insertTerm : In worst case, we scan all existing nodes
So, $O(n)$

* eraseExp : $O(n)$

* evalRecursive : Each recursion evaluates a term
and then calls `powInt()` on exponent

$$\therefore O(n + \sum \log \Delta_i)$$

5 Problem 5

Listing 9: Problem5.h

```
1  /*!
2      Problem 5: Filesystem Flattening [20 points]
3  */
4
5  #include <iostream>
6  using namespace std;
7
8  struct Node {
9      int id; // unique ID (e.g., 1..n)
10     bool isDir; // true = directory, false = file
11     Node* next; // next sibling (or next in flattened list)
12     Node* child; // first child if directory, else nullptr
13     Node(int i, bool d): id(i), isDir(d), next(nullptr),
14         child(nullptr) {}
15 };
16
17 // Append kid at the end of directory dir child-
18 list.
19 void addChild(Node* dir, Node* kid);
20
21 // Append sib after node u in the sibling chain
22 (attach at the end of u's next-chain).
23 void appendSibling(Node* u, Node* sib);
24
25 // Recursively flatten the multilevel list rooted at
26 head into a single-level
27 // preorder list. After flatten, all child pointers in
28 reachable nodes must be nullptr.
29 // Return the head of the flattened list.
30 Node* flatten(Node* head);
31
32 // Print IDs once along next starting at head (e.g.,
33 "10 11 12 ...")
34 // or "EMPTY" if head==nullptr.
35 void printFlat(Node* head);
```

Listing 10: Problem5.cpp

```
1  #include "Problem5.h"
2
3  // Append kid at the end of directory dir child-
4  list.
5  void addChild(Node* dir, Node* kid) {
6      if (dir == nullptr || kid == nullptr) return;
7
8      if (dir->child == nullptr) {
```

```

8         dir->child = kid;
9         kid->next = nullptr;
10        return;
11    }
12
13    Node* cur = dir->child;
14    while (cur->next != nullptr)
15        cur = cur->next;
16
17    cur->next = kid;
18    kid->next = nullptr; // Ensure kid's next is null
19 }
20
21 // Append sib after node u in the sibling chain
22 // (attach at the end of u's next-chain).
23 void appendSibling(Node* u, Node* sib) {
24     if (u == nullptr || sib == nullptr) return;
25
26     Node* cur = u;
27     while (cur->next != nullptr)
28         cur = cur->next;
29
30     cur->next = sib;
31     sib->next = nullptr; // Ensure sib's next is null
32 }
33
34 // Recursively flatten the multilevel list rooted at
35 // head into a single-level
36 // preorder list. After flatten, all child pointers in
37 // reachable nodes must be nullptr.
38 // Return the head of the flattened list.
39 Node* flatten(Node* head) {
40     if (head == nullptr) return nullptr;
41
42     if (head->child != nullptr) {
43         Node* childHead = flatten(head->child);
44         Node* childTail = childHead;
45
46         while (childTail->next != nullptr)
47             childTail = childTail->next;
48
49         Node* oldNext = head->next;
50         childTail->next = oldNext;
51         head->next = childHead;
52         head->child = nullptr;
53     }
54
55     // Flatten the next sibling
56     if (head->next != nullptr)
57         flatten(head->next);

```

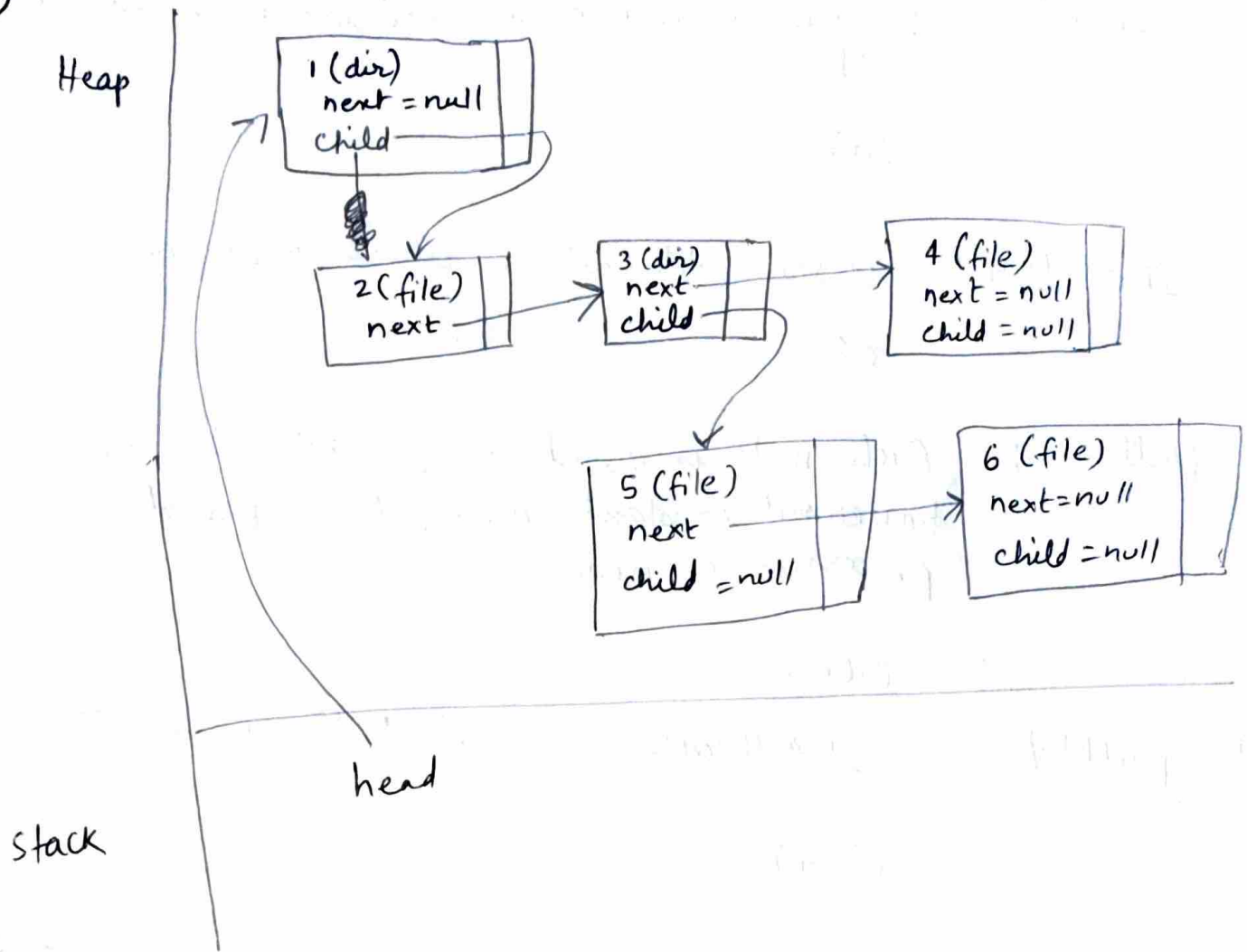
```

55
56     return head;
57 }
58
59 // Print IDs once along next starting at head (e.g.,
60 // "10 11 12 ...")
61 // or "EMPTY" if head==nullptr.
62 void printFlat(Node* head) {
63     if (head == nullptr) {
64         cout << "EMPTY" << endl;
65         return;
66     }
67     Node* cur = head;
68     while (cur != nullptr) {
69         cout << cur->id << " ";
70         cur = cur->next;
71     }
72     cout << endl;
73 }

```

Problem 5:

1.)



3.) After Flatten():

10 → 11 → 12 → 14 → 15 → 16 → 13 → null

The tree is converted in single level list
only next pointers are used to connect the flat sequence.
Each child pointer was cleared during recursion.

4) Complexity:

* addChild : It scans till end of the child list to append new child.

$\therefore O(n)$

* appendSibling : It scans till end to attach new node.

$O(n)$

* flatten : Each node is visited once, each edge is traversed constant times. No repeated processing of nodes.

$\therefore O(n)$

* printFlat : It iterates through flattened list once.

$O(n)$

* Recursion Depth : It corresponds to max no. of nested directories.

$\therefore O(d)$

6 Problem 6

Listing 11: Problem6.h

```
1  /*!
2      Problem 6: Real-Time Task Scheduling with a Ring Buffer
3          Deque [20 points]
4  */
5  #include <iostream>
6  using namespace std;
7
8  struct Task {
9      int id; // unique task identifier
10     // you may add small fields if needed (e.g., stamp), but
11     not STL containers
12 };
13
14 struct DequeRB {
15     Task* a; // circular array on the heap
16     int cap; // capacity (fixed at construction)
17     int head; // index of front element (valid iff count >
18     0)
19     int tail; // index one past the back element (in ring
20     space)
21     int count; // number of elements currently stored
22
23     DequeRB(int C) : cap(C), head(0), tail(0), count(0) {
24         // allocate a[C], set fields for empty deque
25         a = new Task[C];
26     }
27
28     ~DequeRB() {
29         // delete[] a
30         delete[] a;
31     }
32
33     bool empty() const {
34         return count == 0;
35     }
36
37     bool full() const {
38         return count == cap;
39     }
40
41     // push to front/back; pre: not full (ENQ_HIGH/PREEMPT)
42     void pushFront(const Task& t) {
43         head = (head - 1 + cap) % cap; // move head backward
44         in ring
45         a[head] = t;
46         count++;
47     }
```

```

42     }
43     // ENQ_LOW
44     void pushBack (const Task& t) {
45         a[tail] = t;
46         tail = (tail + 1) % cap;
47         count++;
48     }
49
50     // pop from front/back; pre: not empty
51     //DISPATCH()
52     Task popFront() {
53         Task t = a[head];
54         head = (head + 1) % cap;
55         count--;
56         return t;
57     }
58
59     // DROP_LOW()
60     Task popBack () {
61         tail = (tail - 1 + cap) % cap;
62         Task t = a[tail];
63         count--;
64         return t;
65     }
66
67     // accessors; pre: not empty
68     Task& front() {
69         return a[head];
70     }
71     Task& back () {
72         int backIdx = (tail - 1 + cap) % cap;
73         return a[backIdx];
74     }
75 };
76
77 // Simple scheduler facade using the deque.
78 // ENQ_HIGH -> pushFront, ENQ_LOW -> pushBack,
79 // DISPATCH -> popFront (returns dispatched Task id)
80 // PREEMPT(id) -> pushFront(Task{id}),
81 // DROP_LOW() -> popBack().
82 struct Scheduler {
83     DequeRB q;
84
85     Scheduler(int C);
86
87     void ENQ_HIGH(int id) {
88         q.pushFront(Task{id});
89     }
90
91     void ENQ_LOW (int id) {

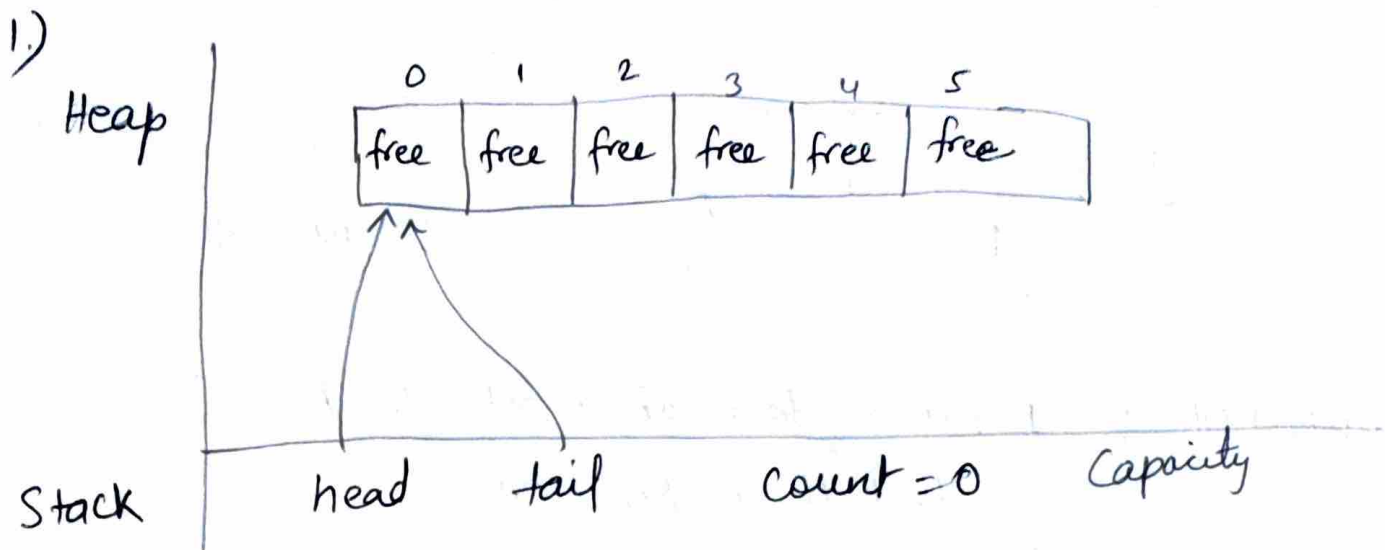
```

```

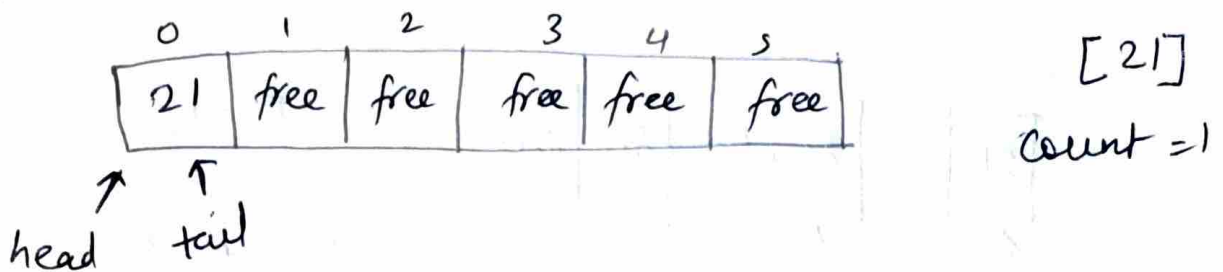
92         q.pushBack(Task{id});
93     }
94
95     int DISPATCH() { // returns id of dispatched Task
96     Task t = q.popFront();
97     return t.id;
98     }
99
100    void PREEMPT(int id) { // place given id back at front
101        q.pushFront(Task{id});
102    }
103
104    void DROP_LOW() { // remove one Task from the back
105        q.popBack();
106    }
107 };
108
109 Scheduler::Scheduler(int C) : q(C) {}

```

Problem 6:



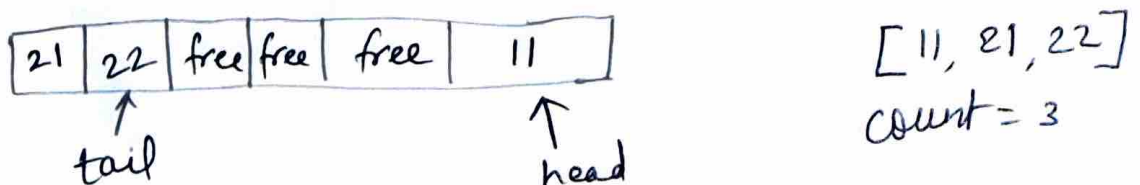
* ENQ_LOW(21): It inserts at the back.



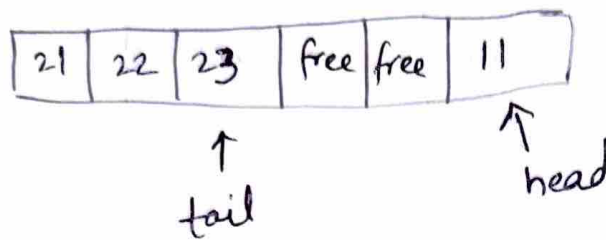
* ENQ_LOW(22): Insert at back, tail moves 0 → 1



* ENQ_HIGH(11): Insert at front, The current head is 0
Moving one position backward in a circular array means wrapping around index 5.
$$\text{Head} = (0 - 1 + 6) \% 6 = 5$$



* ENQ - LOW(23) : Insert at back. Tail moves $1 \rightarrow 2$

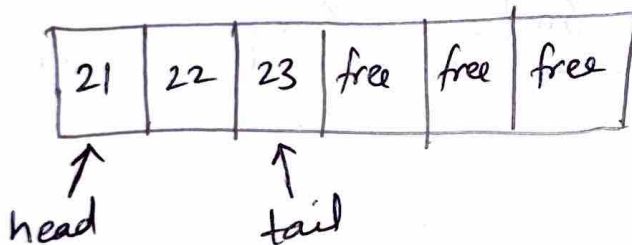


$[11, 21, 22, 23]$

Count = 4

* DISPATCH : Remove task at front (head = 5)
So task 11 is removed.
Then head moves forward.

$$\text{head} = (5 + 1) \% 6 = 0$$



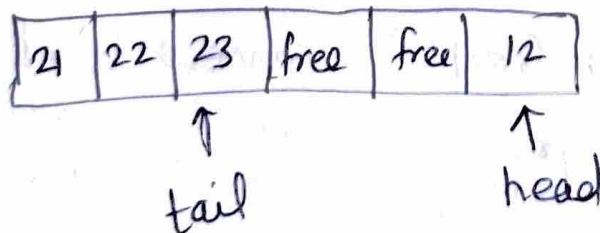
$[21, 22, 23]$

Count = 3

* ENQ - HIGH(12) : Insert at front

Current head = 0, so move backward

$$\text{Head} = (0 - 1 + 6) \% 6 = 5$$



$[12, 21, 22, 23]$

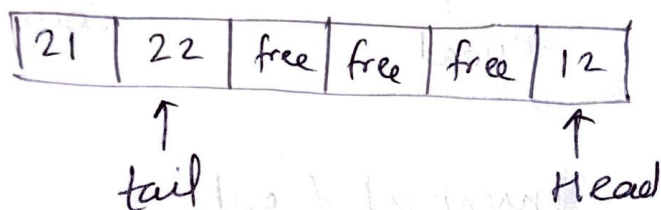
Count = 4

* DROP_LOW() : Remove task at back (tail = 2)

So 23 is removed,

Tail moves backward then

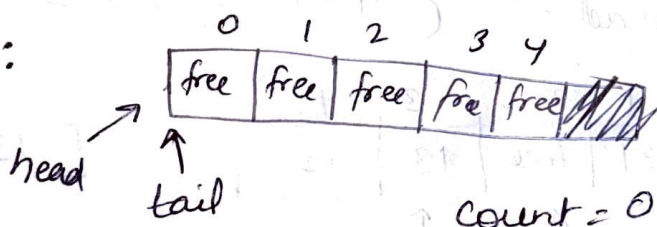
$$\text{Tail} = (2 - 1 + 6) \% 6 = 1$$



[12, 21, 22]

count = 3

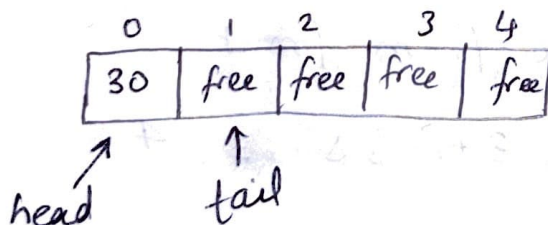
3.) Starting state:



[]

count = 0

* ENQ_LOW(30) : Insert at back



[30]

count = 1

* ENQ_HIGH(12) : Push at front

$$\text{head} = (0 - 1 + 5) \% 5 = 4$$

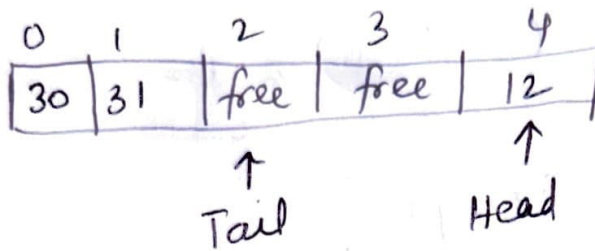


[12, 30]

count = 2

* ENQ-LOW(31) : Insert at back

$$\text{tail} = (1+1) \% 5 = 2$$

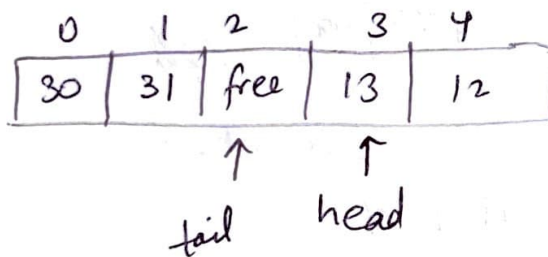


[12, 30, 31]

count = 3

* ENQ-HIGH(13) : Insert at front

$$\text{head} = (4 - 1 + 5) \% 5 = 3$$



[13, 12, 30, 31]

count = 4

* DISPATCH : Remove from front

$$\text{head} = (3+1) \% 5 = 4$$

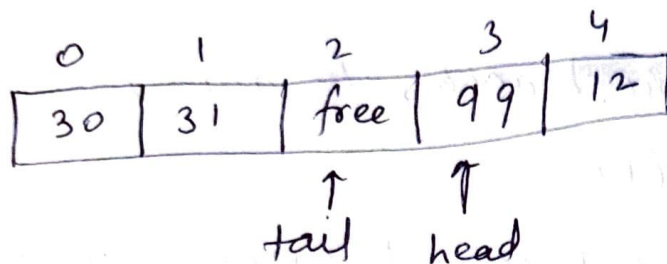


[12, 30, 31]

count = 3

* PREEMPT(99): Push the task at front

$$\text{head} = (4 - 1 + 5) \% 5 = 3$$

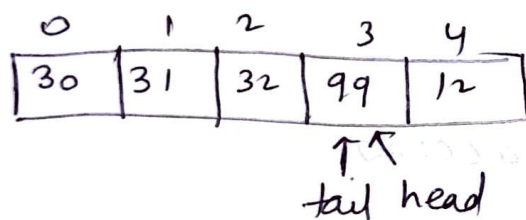


[99, 12, 30, 31]

count = 4

* ENQ_LOW(32): Insert at back

$$\text{tail} = (2 + 1) \% 5 = 3$$



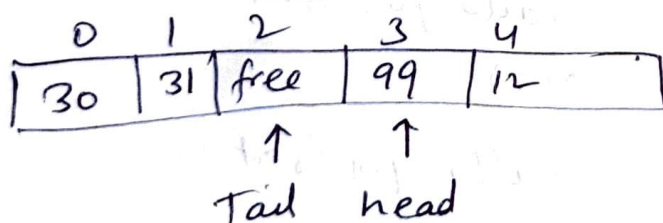
[99, 12, 30, 31, 32]

count = 5

The dequeue is full now

* DROP_LOW(): Remove from back (tail - 1)

$$\text{New tail} = (3 - 1 + 5) \% 5 = 2$$



[99, 12, 30, 31]

count = 4

Returned task = 32

4.) Every dequeue operation uses only index arithmetic and one array access. There is no shifting or resizing so the complexity for all is $O(1)$.

- * $\text{pushFront}()$ — Direct access to $a[\text{head}]$
 $O(1)$
- * $\text{pushBack}()$ — Direct access to array element
 $O(1)$
- * $\text{popFront}()$ — $O(1)$
- * $\text{popBack}()$ — $O(1)$
- * $\text{front}()$ — direct access
— $O(1)$
- * $\text{back}()$ — $O(1)$
- * $\text{ENQ_HIGH}()$ — calls $\text{pushFront}()$
— $O(1)$
- * $\text{ENQ_LOW}()$ — calls $\text{pushBack}()$
— $O(1)$
- * $\text{DISPATCH}()$ — calls $\text{popFront}()$
— $O(1)$
- * $\text{PREEMPT}()$ — calls $\text{pushFront}()$
— $O(1)$
- * $\text{DROP_LOW}()$ — calls $\text{popBack}()$
— $O(1)$

The dequeue allocates exactly 1 heap array.

So the storage required by dequeue is

$$O(c)$$

The additional auxiliary working space beyond that array is

$$O(1)$$

Because only a constant number of variables (head, tail, count) are used.

7 Problem 7

Listing 12: Problem7.h

```
1  /*
2      Problem 7: Particle Simulation: SoA vs. AoS [20 points]
3  */
4
5  // ----- AoS -----
6  struct Particle {
7      double x, y, vx, vy;
8  };
9
10 struct AoS {
11     Particle* p; // array length N on the heap
12     int N;
13 };
14
15 // Allocate AoS of size N and initialize from arrays (x[], y
16 // [], vx[], vy[]).
17 void aosInit(AoS& A, int N,
18             const double* x,
19             const double* y,
20             const double* vx,
21             const double* vy) {
22     A.N = N;
23     A.p = new Particle[N];
24     for (int i = 0; i < N; i++) {
25         A.p[i].x = x[i];
26         A.p[i].y = y[i];
27         A.p[i].vx = vx[i];
28         A.p[i].vy = vy[i];
29     }
30 }
31
32 // Advance positions by dt: x += vx*dt, y += vy*dt for all
33 // particles.
34 void aosStep(AoS& A, double dt) {
35     for (int i = 0; i < A.N; i++) {
36         A.p[i].x += A.p[i].vx * dt;
37         A.p[i].y += A.p[i].vy * dt;
38     }
39 }
40
41 // Compute center of mass: returns (cx, cy) via refs.
42 void aosCenterOfMass(const AoS& A, double& cx, double& cy) {
43     // Since all particles have equal mass, center of mass
44     // is just the average position.
45     double sumX = 0.0;
46     double sumY = 0.0;
```

```

44
45     for (int i = 0; i < A.N; i++) {
46         sumX += A.p[i].x;
47         sumY += A.p[i].y;
48     }
49
50     cx = sumX / A.N;
51     cy = sumY / A.N;
52 }
53
54 // Free AoS heap memory.
55 void aosFree(AoS& A) {
56     delete A.p;
57     A.p = nullptr;
58     A.N = 0;
59 }
60
61 // ----- SoA -----
62 struct SoA {
63     double *x, *y, *vx, *vy; // four arrays length N on the
64     heap
65     int N;
66 };
67 // Allocate SoA of size N and initialize from arrays (x[], y
68 // [], vx[], vy[]).
69 void soaInit(SoA& S, int N,
70             const double* x,
71             const double* y,
72             const double* vx,
73             const double* vy) {
74     S.N = N;
75     S.x = new double[N];
76     S.y = new double[N];
77     S.vx = new double[N];
78     S.vy = new double[N];
79     for (int i = 0; i < N; i++) {
80         S.x[i] = x[i];
81         S.y[i] = y[i];
82         S.vx[i] = vx[i];
83         S.vy[i] = vy[i];
84     }
85 }
86
87 // Advance positions by dt: x[i] += vx[i]*dt, y[i] += vy[i]*
88 // dt.
89 void soaStep(SoA& S, double dt) {
90     for (int i = 0; i < S.N; i++) {
91         S.x[i] += S.vx[i] * dt;
92         S.y[i] += S.vy[i] * dt;

```

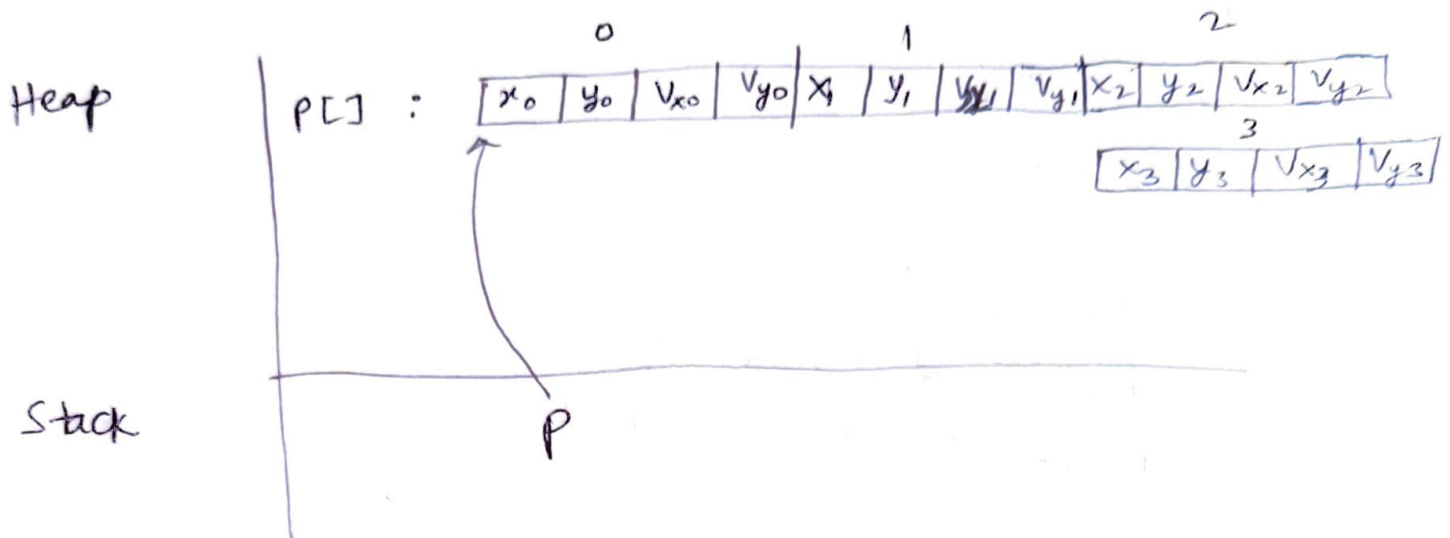
```

91     }
92 }
93 // Compute center of mass: returns (cx, cy) via refs.
94 void soaCenterOfMass(const SoA& S, double& cx, double& cy) {
95     // Since all particles have equal mass, center of mass
96     // is just the average position.
97     double sumX = 0.0;
98     double sumY = 0.0;
99
100     for (int i = 0; i < S.N; i++) {
101         sumX += S.x[i];
102         sumY += S.y[i];
103     }
104
105     cx = sumX / S.N;
106     cy = sumY / S.N;
107 }
108 // Free SoA heap memory.
109 void soaFree(SoA& S) {
110     delete S.x;
111     delete S.y;
112     delete S.vx;
113     delete S.vy;
114
115     S.x = S.y = S.vx = S.vy = nullptr;
116     S.N = 0;
117 }
118
119 // ----- Layout transform (optional helper pair) -----
120 // Copy A -> S (same N); Copy S -> A (same N). Assume
121 // targets allocated.
122 void aosToSoa(const AoS& A, SoA& S) {
123     for (int i = 0; i < A.N; i++) {
124         S.x[i] = A.p[i].x;
125         S.y[i] = A.p[i].y;
126         S.vx[i] = A.p[i].vx;
127         S.vy[i] = A.p[i].vy;
128     }
129 }
130 void soaToAos(const SoA& S, AoS& A) {
131     for (int i = 0; i < S.N; i++) {
132         A.p[i].x = S.x[i];
133         A.p[i].y = S.y[i];
134         A.p[i].vx = S.vx[i];
135         A.p[i].vy = S.vy[i];
136     }
137 }

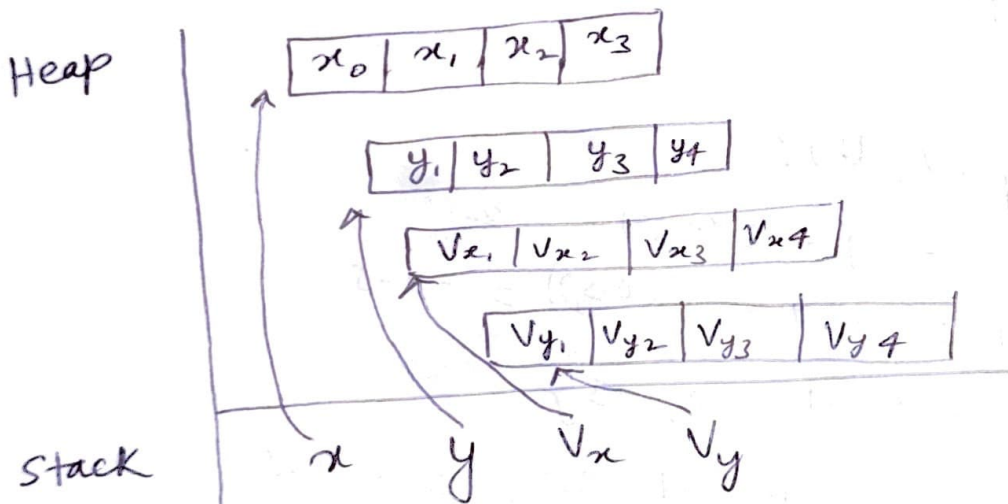
```

Problem 7 :

1.) (a) Array of Structure : for $N=4$



(b) Structure of Arrays :



3) $N=3$ $dt = 0.5$

$$x = [1, -2, 0]$$

$$y = [0, 3, -1]$$

$$V_x = [2, -1, 4]$$

$$V_y = [-3, 0, 1]$$

(a) AoS:

$$P[0] = \{1, 0, 2, -3\}$$

$$P[1] = \{-2, 3, -1, 0\}$$

$$P[2] = \{0, -1, 4, 1\}$$

Memory Picture:

	P[0]				P[1]				P[2]			
P[0]	1	0	2	-3	-2	3	-1	0	0	-1	4	1

After aosStep(A, dt):

P[0]:

$$x_0 = 1 + 2(0.5) = 2$$

$$y_0 = 0 + (-3(0.5)) = -1.5$$

$$P[0] = \{2, -1.5, 2, -3\}$$

Particle 1:

$$x_1 = -2 + (-1 \times 0.5) = -2.5$$

$$y_1 = 3 + (0 \times 0.5) = 3$$

$$P[1] = \{-2.5, 3, -1, 0\}$$

Particle 2:

$$x_2 = 0 + (4 \times 0.5) = 2$$

$$y_2 = -1 + (1 \times 0.5) = -0.5$$

$$P[2] = \{2, -0.5, 4, 1\}$$

AoS memory after step

P[0]				P[1]				P[2]			
2	-1.5	2	-3	-2.5	3	-1	0	2	-0.5	4	1

(b) SoA : Before step

$$x: \begin{array}{|c|c|c|} \hline 1 & -2 & 0 \\ \hline \end{array}$$

$$y: \begin{array}{|c|c|c|} \hline 0 & 3 & -1 \\ \hline \end{array}$$

$$V_x: \begin{array}{|c|c|c|} \hline 2 & -1 & 4 \\ \hline \end{array}$$

$$V_y: \begin{array}{|c|c|c|} \hline -3 & 0 & 1 \\ \hline \end{array}$$

After step:

$$\begin{array}{|c|c|c|} \hline 2 & -2.5 & 2 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|} \hline -1.5 & 3 & -0.5 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|} \hline 2 & -1 & 4 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|} \hline -3 & 0 & 1 \\ \hline \end{array}$$

(c) For equal-mass particles, centre of mass is the avg position.

$$C_x = \frac{x_0 + x_1 + x_2}{3}$$

$$C_y = \frac{y_0 + y_1 + y_2}{3}$$

After the step: $x[] = \{2, -2.5, 2\}$

$$y[] = \{-1.5, 3, -0.5\}$$

$$C_x = \frac{2 - 2.5 + 2}{3} = 0.5$$

$$C_y = \frac{-1.5 + 3 - 0.5}{2} = 0.33$$

Both layouts produce exactly same ~~mass~~ ^{Centre of mass} :

AoS : (0.5, 0.33)

SoA : (0.5, 0.33)

4) Both AoS and SoA have same time complexity but their memory access pattern differ.

<u>Function</u>	<u>Time</u>	<u>Aux Space</u>
aosInit	$O(N)$	$O(1)$
aosStep	$O(N)$	$O(1)$
aosCenter of Mass	$O(N)$	$O(1)$
aosFree	$O(1)$	$O(1)$
soaInit	$O(N)$	$O(1)$
soaStep	$O(N)$	$O(1)$
soaCentre of Mass	$O(N)$	$O(1)$
soaFree	$O(1)$	$O(1)$
aos To Soa	$O(N)$	$O(1)$
soa To Aos	$O(N)$	$O(1)$

All init, step, centre of mass & conversion fn take $O(N)$ as they iterate over all N particles.

The free functions use constant number of delete operations.